

MacroShock: Complete Package Documentation

Reflects package version 1.0.1*

Patricio Arias[†]

July 2026

Abstract

This document is a complete technical reference for the `macroshock` Python package (v1.0.1): every input, every output, and every method of the `SVAR` class, together with a module-by-module description of the ten files that implement the estimation, identification, inference and plotting pipeline. It is meant to be read alongside the code, and is aimed at users who want a precise description of what each parameter does and what each method returns, and at maintainers who need to understand how the modules fit together. Section 1 gives an installation and architecture overview; Section 2 documents the `SVAR` class method by method; Section 3 documents the ten underlying modules function by function; Section 4 collects every input/output table; Section 5 lists the caveats and known issues found while writing this reference, so they can be triaged before the next release. Boxes marked ✓ record inconsistencies from earlier drafts of this reference that the code has since fixed; boxes marked △ are still open.

Contents

1	Overview	3
1.1	Installation	3
1.2	Package architecture	3
2	The SVAR class	4
2.1	Constructor processing steps	4
2.2	<code>SVAR.YX()</code>	4
2.3	<code>SVAR.VAR()</code>	4
2.4	<code>SVAR.S()</code>	5
2.5	<code>SVAR.run(show=False, IRbands='pair', FCbands='many', showIR=False, showFC=False, showVD=False)</code>	5
2.6	<code>SVAR.summary()</code>	5
2.7	<code>SVAR.ImpulseResponse(shock, show=True, bands='pair', reps=None, steps=None, impact=None)</code>	5
2.8	<code>SVAR.Forecast(show=True, bands='many', horizon=None, past=None, reps=None)</code>	6
2.9	<code>SVAR.VarianceDecomp(shock, show=True, bands='pair', steps=None, reps=None)</code>	6
2.10	<code>SVAR.stability(show=False)</code>	6
2.11	<code>SVAR.HistoricalDecomp()</code>	7
2.12	The SVEC and LP classes	7

*Technical reference manual. For the accompanying academic paper (theory, replications, references) see *MacroShock: Python Package for SVAR Analysis of Time Series*. Package repository: <https://github.com/patricioariaslpz/macroshock>.

[†]Instituto Tecnológico Autónomo de México, patricio.arias@itam.mx

3	Module reference	7
3.1	data.py	7
3.2	stats.py	7
3.3	var.py	8
3.4	identification.py	8
3.5	irf.py	9
3.6	fevd.py	9
3.7	fc.py	10
3.8	bootstrap.py	10
3.9	plotting.py	11
4	Tables	12
5	Known issues	17

1 Overview

1.1 Installation

MacroShock is distributed through the Python Package Index (PyPI):

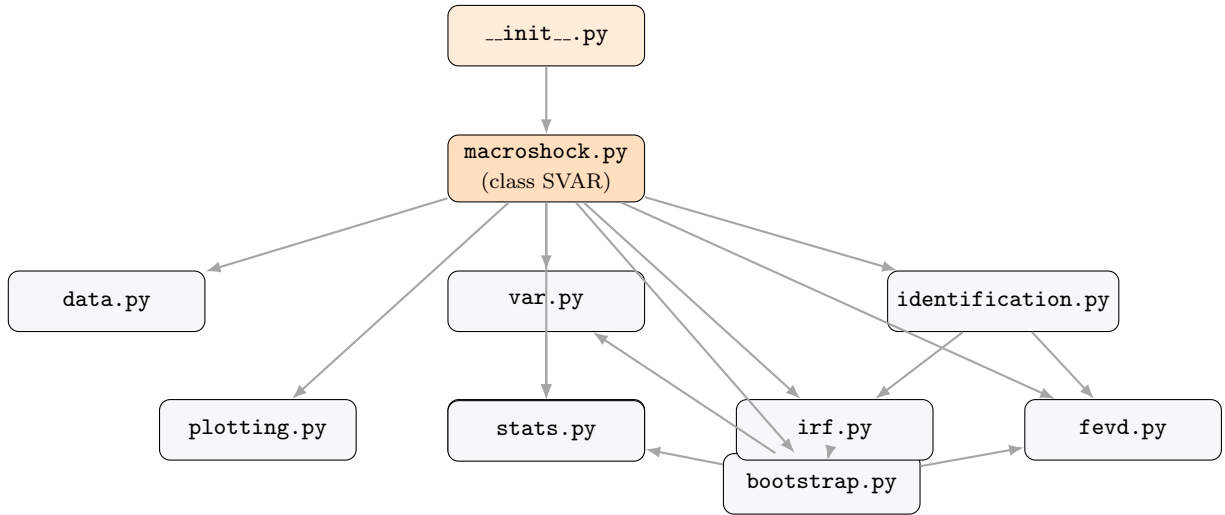
```
pip install macroshock
```

It requires Python 3.9 or later and depends on *Numpy*, *SciPy*, *pandas* and *Matplotlib*, installed automatically. The main class is imported with

```
from macroshock import SVAR
```

1.2 Package architecture

The package has eleven files: ten stateless, low-level modules and one orchestrating class, **SVAR**, which keeps the model's state and delegates each numerical step to the modules below it.



- **data.py** – builds Y , X (deterministic terms, lags, exogenous variables, seasonal dummies) and the restriction matrix R .
- **var.py** – estimates the restricted VAR by feasible GLS, and computes standard errors, t -statistics, p -values and information criteria, including FPE.
- **stats.py** – Student's t CDF and two-sided p -values (SciPy).
- **identification.py** – the four structural identification schemes: short-run, long-run, sign restrictions, and instrumental variables.
- **irf.py** – impulse-response computation and bootstrap-draw summarization (mean/median/percentiles).
- **fevd.py** – forecast error variance decomposition.
- **fc.py** – conditional point forecasts.
- **bootstrap.py** – normal and wild bootstrap resampling for IR, FC and VD confidence bands.
- **plotting.py** – Matplotlib plotting routines for IRF, forecast and FEVD.
- **macroshock.py** – the **SVAR** class (plus the scaffolded **SVEC** and **LP** classes).
- **__init__.py** – exposes **SVAR** at the package's top level.

△ Note: No module in the list above imports the **SVAR** class; all of them operate on a `model` argument by duck typing. This makes each module independently testable, but also means there is no validation that a given object actually carries the attributes a function expects before it is called.

2 The SVAR class

The SVAR class is created with

```
model_name = SVAR(input)
```

Endogenous data can be passed either as a `pandas` DataFrame (in which case `variables` selects the relevant columns) or as a raw Numpy array. The complete list of constructor inputs is given in Table 1.

2.1 Constructor processing steps

Beyond simply storing its arguments, `--init--` performs the following steps, in order:

1. Copies every parameter to an attribute of `self`, including `self.R = np.array(R)` – if `R` is left at its default of `None`, this produces a 0-dimensional object array holding `None`, which is harmless because `SVAR.YX()` always reassigns `self.R` before it is used.
2. Builds `method_dict` (human-readable names used by `summary()`) and `self.n_vars = len(self.variables)`.
3. Defaults `titles` to `variables` if not given, and fills `rescaling/transformation` with neutral per-variable defaults if `None`.
4. If `data` is a DataFrame: extracts `iv` as named columns first (only if `method='IV'`), then overwrites `self.data` with the columns in `variables`.
5. Processes exogenous data (`data_exogenous/variables_exogenous`): if `data_exogenous` is a DataFrame, extracts the named columns and records `n_obsex, n_ex`; if `data_exogenous` is `None`, sets `n_ex=0` and defaults `variables_exogenous` to an empty placeholder list.
6. Transforms the data column by column according to `transformation[i] ∈ {'diff', 'logdiff', 'log', None}`, applying `rescaling[i]` in *every* branch (multiplicatively). If any variable uses `'diff'` or `'logdiff'`, the first row is dropped from *all* columns to keep them aligned.
7. If `method='IV'`: reshapes `iv` to 2-D if needed and records `n_obsi, n_iv`.

✓ **Fixed in v1.0.1: Rescaling asymmetry.** Earlier drafts of this package multiplied by `rescaling[i]` for the `'log'`, `'logdiff'` and no-transformation branches, but *not* for `'diff'`, leaving a plain first difference unscaled. In v1.0.1, `rescaling` is applied uniformly across all four branches.

2.2 SVAR.YX()

Delegates to `data.YX(self)` (Section 3.1); builds `self.Y`, `self.X`, `self.R` and, if applicable, seasonal dummy attributes.

2.3 SVAR.VAR()

Delegates to `var.estimate_var(self.Y, self.X, self.R)` (Section 3.3), copies every field of the result onto `self` (including the new `self.n_params`), and then *itself* constructs the companion matrix

$$F = \begin{bmatrix} \Phi_1 & \Phi_2 & \cdots & \Phi_k \\ I_n & \mathbf{0} & \cdots & \mathbf{0} \\ \vdots & & \ddots & \vdots \\ \mathbf{0} & \cdots & I_n & \mathbf{0} \end{bmatrix}$$

by slicing `self.beta[:, self.const : self.const + self.lags*self.n_vars]` – i.e. the block of columns immediately after the deterministic terms, which is where `data.py` always places the lag coefficients regardless of whether exogenous or dummy columns follow them. `var.estimate_var()` itself never builds F (see Section 3.3); this is the only place in the codebase that builds it *correctly* – see the divergent, incorrect re-implementation in `bootstrap.py` (Section 3.8 and Section 5).

2.4 SVAR.S()

Dispatches to one of the four identification functions in `identification.py` (Section 3.4) according to `self.method`, and always sets `self.structural = True` at the end – even if `self.method` is not a recognized value, in which case no branch executes and `self.B` is never created.

△ Note: An unrecognized `method` does not raise an error here; it silently leaves the model marked `structural=True` with no `self.B`. The resulting `AttributeError` surfaces later, in an unrelated method (`summary()`, `ImpulseResponse()`, etc.), which makes the root cause harder to spot.

For `method='signs'`, `S()` additionally caches 24 shock-indexed attributes needed later by `ImpulseResponse()` and `VarianceDecomp()`, listed in Table 2.

2.5 SVAR.run(show=False, IRbands='pair', FCbands='many', showIR=False, showFC=False, showVD=False)

Orchestrates the full pipeline: `YX()` → `VAR()` → `S()` (if `method` is not `None`) → `summary()`, then optionally:

- `showIR=True`: calls `ImpulseResponse(k+1, show=show, bands=IRbands)` for every shock $k = 0, \dots, n_{vars} - 1$.
- `showFC=True`: calls `Forecast(bands=FCbands)` once.
- `showVD=True`: calls `VarianceDecomp(k+1, show=show, bands=IRbands)` for every shock $k = 0, \dots, n_{vars} - 1$.

✓ **Fixed in v1.0.1:** Earlier drafts of `run()` only computed `VarianceDecomp()` for shock 1 when `showVD=True`, unlike `showIR` which already iterated over every shock. Both now iterate identically.

△ Note: `run()` has no separate `VDbands` parameter: the variance-decomposition plots triggered by `showVD=True` reuse whatever was passed as `IRbands`, even though `VarianceDecomp()`'s own `bands` convention is really about the FEVD stacked-area plot, which does not use `bands` at all for its visual (see Section 2.9) – so this argument is effectively inert for the FEVD plot in practice, only affecting the dict key naming path internally.

The attributes populated on the model object once `run()` completes are listed in Table 3; access any of them with `model_name.output_name`.

2.6 SVAR.summary()

Prints (and stores as the string `self.summary`) the coefficient table with standard errors, *t*-statistics and *p*-values for every equation, the information criteria (including the number of free parameters, `self.n.params`, and FPE), and – once the model is structural – the identification method, rejection/acceptance counts, instrument strength (`method='IV'`), and the estimated *B* matrix (labeled "Matrix B (median)" when `method='signs'`, which is accurate: see Section 3.4). Takes no input.

2.7 SVAR.ImpulseResponse(shock, show=True, bands='pair', reps=None, steps=None, impact=None)

Computes the impulse-response function of shock `shock` together with confidence bands, and optionally plots it.

- `reps`, `steps`, `impact` default to `self.reps`, `self.steps`, `self.impact` respectively, but – unlike earlier drafts – are purely *local* variables: passing an override no longer mutates `self.steps` (or any other model attribute) as a side effect.
- For `method ≠ 'signs'`: calls `bootstrap_svar(self, typ='IR', shock=shock, reps=reps, steps=steps)`.
- For `method='signs'`: slices the cached statistics from Table 2 instead, with no new sampling.

✓ **Fixed in v1.0.1:** Earlier drafts overwrote `self.steps/self.impact` permanently whenever a per-call override was passed, so a single `ImpulseResponse(steps=...)` call could silently change

the horizon used by later, unrelated `Forecast()`/`VarianceDecomp()` calls. `reps/steps/impact` are now local to the call.

△ Note: `impact` is still not actually wired through. The local `impact` variable is computed (from the override or `self.impact`) but is *never passed* to the `bootstrap_svar()` call above – `bootstrap_svar()` has its own `impact` parameter, defaulting to 1.0, which is what actually gets used regardless of what `self.impact` or the `impact=` override say. The `'signs'` branch does not use `impact` at all (the cached IRFs in `identify_signs()` are always computed with a unit shock).

△ Note: **Output-dictionary bug for method $\neq$ `'signs'`.** The returned dict maps key 99.5 to `self.ir_97` (a duplicate of key 97.5) instead of `self.ir_99`; `self.ir_99`, which is correctly computed by `compute_bands_from_draws()`, is silently unused. The `'signs'` branch does not have this bug: its 99.5 key correctly maps to `self.ir_99`.

Input and output are given in Tables 4 and 6.

2.8 `SVAR.Forecast(show=True, bands='many', horizon=None, past=None, reps=None)`

Computes the conditional point forecast together with confidence bands.

- `horizon`, `past`, `reps` default to `self.horizon`, `self.past`, `self.reps`.
- For method $\neq$ `'signs'`: also computes an unused deterministic point forecast `self.fc_point` (via `compute_forecast(self, self.F)`, which reads `self.horizon/self.past` directly rather than the local overrides – see warning below), then calls `bootstrap_svar(self, typ='FC', horizon=horizon, past=past, reps=reps)` for the actual bootstrap bands.
- For method `'signs'`: a single deterministic path via `compute_forecast(self, self.F)`, with no real dispersion – all nine output keys are identical.

△ Note: `horizon/past` overrides are ignored under method `'signs'`. The `'signs'` branch calls `compute_forecast(self, self.F)` directly, which always reads `self.horizon/self.past` (see `fc.py`, Section 3.7) rather than the local `horizon/past` arguments – so a call like `Forecast(horizon=12)` under a sign-restricted model silently forecasts `self.horizon` periods instead of 12. The same is true of `self.fc_point` under non-`'signs'` methods: it is computed but never placed in the output dictionary or used by the plot, so it is effectively dead weight, and if it *were* used it would also ignore the local `horizon/past` overrides.

Input and output are given in Table 7.

2.9 `SVAR.VarianceDecomp(shock, show=True, bands='pair', steps=None, reps=None)`

Computes the forecast error variance decomposition attributable to `shock`. `shock` is now a required positional argument here too (no default), matching `ImpulseResponse()`. Both branches (method `'signs'` or not) first compute `self.vd_det`, the deterministic decomposition of the current `self.B` – this is the only object the stacked-area plot uses, via `plot_variance_decomp_svar(self, shock)` (Section 3.9). Input and output tables are Tables 5 and 6.

✓ **Fixed in v1.0.1:** Earlier drafts gave `shock` a default of 1 here (asymmetric with `ImpulseResponse()`, where it was required). It is now required in both methods.

△ Note: Each call to `VarianceDecomp(shock=k)` under a bootstrapped method re-runs `reps` full replications from scratch, even though `forecast_error_variance_decomposition()` computes the complete decomposition (every shock, every variable) inside each replication and `bootstrap.py` discards every column except `shock-1`. Reporting all n shocks of a model therefore costs $n \times$ the necessary bootstrap work. The new `reps` override lets you lower the cost per call, but does not remove the underlying redundancy.

2.10 `SVAR.stability(show=False)`

Checks that every eigenvalue of the companion matrix F lies strictly inside the unit circle (a necessary condition for the model's Wold representation to be well defined) and returns a Boolean. Input and output are given in Table 8.

2.11 SVAR.HistoricalDecomp()

return 0. Not implemented; a pure placeholder for the historical decomposition described in the accompanying paper.

2.12 The SVEC and LP classes

Both take only `__init__(self, data)` and store `self.data`. No other method exists yet; both are road-map placeholders (Structural Vector Error Correction and Local Projections, respectively).

3 Module reference

This section documents each of the ten low-level modules that **SVAR** delegates to. Every function here is stateless: it receives arrays (or a `model`-like object) and returns arrays or a `dataclass`, without depending on the **SVAR** class.

3.1 data.py

`YX(model) -> YXR`

Builds Y (regressand) and X (deterministic terms, lags, exogenous variables, seasonal dummies) from `model.data`, and the restriction matrix R from `model.restrictions`. Writes its results directly onto `model` and also returns them packaged in `YXR(Y, X, R)`.

Algorithm.

1. $Y = \text{model.data}[\text{lags}, :]$; X stacks the `lags` lags of `model.data`, in order $[\mathbf{y}_{t-1}, \dots, \mathbf{y}_{t-k}]$.
2. Prepends constant/trend columns per `const`.
3. Concatenates exogenous data (`model.data_exogenous`), recorted to the last `n_obs` rows if needed.
4. If `DUM='Q'` or `'M'`: builds the full dummy matrix `model.D` (including the forecast horizon, so `fc.py` has future dummies available), recorted to $n_{dum} = 3$ or 11 columns (one category dropped to avoid collinearity with the constant), and concatenates it.
5. Builds R : `None` $\rightarrow$ identity (no restrictions); `'zeros'` $\rightarrow$ interprets `model.R` as an $n \times n$ variable-inclusion mask and drops the corresponding columns for every lag simultaneously; `'custom zeros'` $\rightarrow$ interprets `model.R` as a full `vec(beta)`-sized matrix and drops each zero entry individually.

Column order of X : `[deterministic, lags, exogenous, dummies]`. This order matters: `SVAR.VAR()` and `summary()` both slice `beta/X` assuming it, and `bootstrap.py`'s inline reconstruction of the companion matrix assumes something different (see Section 3.8 and Section 5).

△ Note: `model.R` is read with two different semantics depending on `model.restrictions` – an $n \times n$ mask for `'zeros'`, or a full `vec(beta)`-sized array for `'custom zeros'`. Easy to confuse when building the input.

3.2 stats.py

Requires SciPy (raises `ImportError` at import time otherwise). `t_cdf(t, df) / t_pvalue(t_stat, df)`

`t_cdf` is the Student's t CDF via `scipy.stats.t.cdf`. `t_pvalue` is the two-sided p -value $p = 2 \cdot \text{sf}(|t|, df)$ via `scipy.stats.t.sf` (survival function, for numerical stability in the tails) – this is what `var.py` uses for every coefficient's p -value.

3.3 var.py

`estimate_var(Y, X, R) -> VARResults`

Estimates a restricted VAR by feasible GLS (Lütkepohl 2005, Ch. 5).

1. Unrestricted OLS pass to get a preliminary $\hat{\Sigma}_u$: $\beta_{OLS} = (X'X)^{-1}X'Y$.
2. Restricted GLS: $\hat{\gamma} = [R'(X'X \otimes \Sigma_u^{-1})R]^{-1}R'\text{vec}(\Sigma_u^{-1}Y'X)$, $\hat{\beta} = \text{vec}^{-1}(R\hat{\gamma})$ (order 'F').
3. Recomputes $\hat{u} = Y - X\hat{\beta}'$ and $\hat{\Sigma}_u$ with the restricted β .
4. Standard errors from $[R'(X'X \otimes \Sigma_u^{-1})R]^{-1}$ propagated through R ; t -stats and p -values.
5. Information criteria from the full Gaussian log-likelihood, with $n_{\text{params}} = \text{R.shape}[1]$ free parameters: $AIC = 2n_{\text{params}} - 2\ell$, $BIC = n_{\text{params}} \log T - 2\ell$, $HQIC = 2n_{\text{params}} \log \log T - 2\ell$.
6. $FPE = \det(\hat{\Sigma}_u) \cdot \left(\frac{T+k}{T-k}\right)^n$ (Lütkepohl 2005, eq. 4.3.9), where $k = \text{n_cols}$ is the number of regressors per equation and $n = \text{n_vars}$.

Returns `VARResults(beta, sigma_u, resid, Y_hat, ll, aic, bic, hqic, det, fpe, beta_std, tstat, pvalue, n_obs, n_vars, n_cols, n_params)`.

✓ **Fixed in v1.0.1:** Earlier drafts left `fpe` hardcoded at 0.0 (not implemented). It is now computed as above.

△ **Note:** p -values still use $df = T - k$ (`n_obs - n_cols`) uniformly for every coefficient, not adjusted for the number of restrictions actually imposed via R – a restricted equation with fewer free parameters than `n_cols` gets the same (understated) degrees of freedom as an unrestricted one.

△ **Note:** `VARResults` has no `F` field at all in v1.0.1 (it was removed, not just left `None`). `estimate_var()` never builds the companion matrix; that happens exclusively in `SVAR.VAR()` (Section 2), using `self.const/self.lags` to slice the correct block of `self.beta`. `bootstrap.py` reconstructs it again inline, but with a different (and, when exogenous or dummy columns are present, incorrect) slicing rule – see Section 5.

3.4 identification.py

All four functions return a `StructuralResults(B, method, n_rejected, m_accepted, corr, extra)`.

`identify_short_run(sigma_u, matrix=None)`

No mask: $B = \text{Chol}(\Sigma_u)$. With an $n \times n$ zero-restriction mask (1=free, 0=zero): solves B by BFGS minimization of $\|\Sigma_u - BB'\|_F^2$ subject to the mask, after checking the order condition $\sum(\text{mask}) \leq n(n+1)/2$.

`identify_long_run(sigma_u, F, lags, matrix=None)`

Builds $M_1 = (I_n - \sum_s \Phi_s)^{-1}$ and $\Sigma_C = M_1 \Sigma_u M_1'$. No mask: $C = \text{Chol}(\Sigma_C)$ (standard Blanchard-Quah). With a mask: same BFGS optimization as above, applied to C and Σ_C . Recovers $B = M_1^{-1}C$.

`identify_signs(sigma_u, F, matrix_signs, steps_signs, steps, reps, alpha)`

Sign-restriction identification via accept/reject sampling of orthonormal rotations (Rubio-Ramírez, Waggoner & Zha, 2010): draws Q from a QR decomposition of a random normal matrix, proposes $B_p = \text{Chol}(\Sigma_u)Q$, flips columns to satisfy contemporaneous sign restrictions, then checks horizontal (IRF) sign restrictions over `steps_signs` horizons per shock. Accepted draws feed both the IRF set (IRL) and the FEVD set (FEVDL), from which mean/median/percentile statistics are computed via the `_zero_stat_dicts()` helper (see below). Returns **B=the median** of the accepted matrices (`B_median`); the mean is no longer even computed.

✓ **Fixed in v1.0.1:** Earlier drafts returned `B=B_mean` while every other part of the codebase (`SVAR.summary()`'s "Matrix B (median)" header, `VarianceDecomp()`'s comments) described it as

the median. `identify_signs()` now returns the median and the mean is not computed at all, so the code and its own labels agree.

✓ **Fixed in v1.0.1: Early-abort branch (too many rejections).** Earlier drafts returned an extra dict missing the `ir_*/vd_*` keys that `SVAR.S()` unconditionally unpacks, which would raise a `KeyError` if the 100,000-rejection abort ever triggered. A new helper, `_zero_stat_dicts(steps, n_vars, extra_percentiles)`, now builds a zero-filled baseline for all 16 IRF and 10 FEVD statistics, keyed 1..`n_vars`; this baseline is used both in the abort branch and as the starting point of the normal post-loop statistics build (which also fixes the separate, subtler bug where a single shock with zero accepted draws would crash `np.stack([])` even on a successful run).

△ **Note:** The `_high/_low` percentile convention is still swapped here (`_high` uses the low percentile and vice versa); this is compensated on read inside `SVAR.ImpulseResponse()` and `SVAR.VarianceDecomp()`.

`identify_iv(sigma_u, resid, iv, data_array)`

Two-stage proxy-SVAR identification per instrument: first-stage regression of the residual on the instrument, second-stage regression of the fitted first-stage values on the other residuals, normalizing $b_{11} = 1$, then rescaling the identified column by $1/\|\mathbf{q}\|$ where $C\mathbf{q} = \mathbf{b}_1$ and $C = \text{Chol}(\Sigma_u)$, so that $\mathbf{b}_1^{\text{norm}} = \mathbf{b}_1/\|\mathbf{q}\|$ is a valid orthonormal-rotation column ($\mathbf{b}_1^{\text{norm}} = C(\kappa\mathbf{q})$ with $\kappa = 1/\|\mathbf{q}\|$).

✓ **Fixed in v1.0.1:** Earlier drafts multiplied by $\|\mathbf{q}\|$ instead of dividing (`B[:, j] = B[:, j] * v`), which does not satisfy $\|\kappa\mathbf{q}\| = 1$ and was inconsistent with `bootstrap.py`'s inline re-identification under `method='IV'`, which already divided correctly. `identify_iv()` now divides (`B[:, j] = B[:, j] / v`), so the point estimate of `SVAR.B` and its bootstrap confidence bands are built with the same normalization.

3.5 irf.py

`impulse_response(B, F, shock, impact=1.0, steps=10)`

$IR_{shock}(h) = \Psi_h B \mathbf{e}_{shock} \cdot \text{impact}$, Ψ_h = the $n \times n$ upper-left block of F^h .

✓ **Fixed in v1.0.1:** Earlier drafts called `np.linalg.matrix_power(F, h)` from scratch at every horizon. The function now accumulates $F^h = F^{h-1} \cdot F$ incrementally across the loop, replacing $O(\text{steps})$ independent matrix-power calls with $O(\text{steps})$ matrix multiplications – meaningful given this function runs once per bootstrap replication.

`compute_bands_from_draws(draws, alpha)`

From `draws[var]` of shape `(reps, T)`, computes `'point'/'mean'` (identical), `'low'/'high'` (correctly-ordered percentiles $\alpha/2, 100 - \alpha/2$) and the fixed percentiles `q0_5, q2_5, q5, q95, q97_5, q99_5`. Used by `ImpulseResponse()/VarianceDecomp()` for every method except `'signs'`.

3.6 fevd.py

`forecast_error_variance_decomposition(B, F, steps=10)`

Implements

$$FED_{ijh} = \frac{\sum_{m=0}^h (\mathbf{e}_i' \Phi^m B \mathbf{e}_j)^2}{\sum_{k=1}^n \sum_{m=0}^h (\mathbf{e}_i' \Phi^m B \mathbf{e}_k)^2}$$

by precomputing $\theta_h = \Psi_h B$ for every horizon (accumulating F^h incrementally, same optimization as `irf.py` above), then normalizing cumulative squared responses per variable. Returns a dict keyed by 1-based variable index, each value shape `(steps, n_vars)` (columns = shocks).

3.7 fc.py

`compute_forecast(modelo, F)`

Deterministic recursive forecast from the companion form, starting at the last `lags` observations of `modelo.Y`, adding deterministic terms and dummy contributions at each step. Always reads `modelo.horizon/modelo.past` directly – it has no parameters of its own to override them (see the warning on `SVAR.Forecast()`, Section 2).

✓ **Fixed in v1.0.1:** Earlier drafts used $(i + \text{lags}) * \text{beta_trend}$ for `const=2` but plain $i * \text{beta_trend}$ / $i**2 * \text{beta_sqrd}$ (no `+lags` offset) for `const=3`, an asymmetry likely to produce a discontinuity in the trend at the historical/forecast boundary under `const=3`. Both branches now use the same $i * \text{beta_trend}$ convention with no offset.

3.8 bootstrap.py

`bootstrap_svar(model, typ, shock=1, reps=100, impact=1.0, steps=30, horizon=5, past=24)`

Generates `reps` replications: resamples residuals (normal or wild), reconstructs a simulated series recursively, re-estimates the VAR (`estimate_var`) and re-identifies B_{sim} inline for `method` equal to `'short'`, `'long'` or `'IV'`, then stores the requested statistic (`typ` equal to `'IR'`, `'FC'` or `'VD'`).

✓ **Fixed in v1.0.1: Sign-restriction models.** Earlier drafts silently accepted `method='signs'` and looped forever (the replication counter never advanced, since the re-identification block only covers `'short'/'long'/'IV'`). `bootstrap_svar()` now raises `ValueError("No bootstrap in sign restriction SVAR")` immediately if `model.method == 'signs'`.

△ **Note: Defaults no longer come from the model.** `reps`, `steps`, `horizon`, `past` and `impact` are now plain function parameters with their own hardcoded defaults (100, 30, 5, 24, 1.0), independent of `model.reps/model.steps/model.horizon/model.past/model.impact` (whose own defaults are 1000/60/6/12/1.0). Every call site inside `SVAR` (`ImpulseResponse()`, `Forecast()`, `VarianceDecomp()`) explicitly forwards the model's (possibly overridden) values, so this only matters if `bootstrap_svar()` is called directly rather than through the `SVAR` class – doing so silently uses far fewer replications and a shorter horizon than the model was configured with.

△ **Note: Companion-matrix construction diverges from `SVAR.VAR()` and can be wrong.** Inside the replication loop, `topsim = betasim[:, -(model.n_vars * model.lags):]` grabs the *last* `n_vars*lags` columns of the simulated coefficients to build `Fsim`. This is only correct if the lag block is literally the last thing in `X`'s column order. But `data.py` builds `X` as `[deterministic, lags, exogenous, dummies]` (Section 3.1) – so whenever a model has `data_exogenous` and/or `DUM` set, the lag block is *not* at the end, and this slice silently grabs the wrong columns. `SVAR.VAR()` avoids this by slicing from the front instead (`self.beta[:, self.const : self.const + lags*n_vars]`, Section 2), which is correct regardless of what follows the lag block. **Practical impact:** for any model with exogenous regressors or seasonal dummies, every bootstrap replication's companion matrix `Fsim` is built from the wrong coefficients, which corrupts the confidence bands returned by `ImpulseResponse()`, `Forecast()` and `VarianceDecomp()` for that model – silently, with no error or warning. Models with no exogenous data and no seasonal dummies are unaffected, since in that case the lag block *is* the last block.

△ **Note:** Duplicates the X/Y construction logic of `data.py` inline rather than reusing it, which is how the divergence above was able to happen in the first place – any future change to `data.py`'s column order needs a matching manual update here.

3.9 plotting.py

Three functions, all agnostic to `model.method`:

`plot_irf_svar(model, shock)`

Grid of $1 \times n_{vars}$ subplots, one per endogenous variable, showing the IRF to the given structural `shock` with either a single **'pair'** confidence band or three nested **'many'** bands.

`plot_forecast_svar(model)`

Same subplot grid, showing the forecasted path (with bands) alongside `model.past` periods of history, a vertical dashed line marking the historical/forecast boundary.

`plot_variance_decomp_svar(model, shock)`

A *single* stacked-area chart, from `model.vd_det[shock]`, showing the share of the forecast-error variance of **one variable** attributed to every structural shock, across horizons. Requires `model.vd_det`, `model.variables`, `model.shocks`, `model.steps` (populated by `VarianceDecomp()`).

△ Note: The `shock` argument here indexes a *variable*, not a structural shock, despite the name: `var_name = model.variables[shock - 1]` and the plot title/data are `model.vd_det[shock]` – i.e. *variable* number `shock`'s FEVD, decomposed across *all* structural shocks (each shown as one color band in the stack). This mirrors the `shock` argument of `SVAR.VarianceDecomp()` that calls it (`plot_variance_decomp_svar(self, shock)`), where `shock` genuinely does select a structural shock for the returned statistics dict – but the *plot* it triggers shows that same index applied to `self.variables` instead. In a model where shock k is not conceptually "the shock for variable k " (e.g. under `method='signs'` or `'IV'` with an arbitrary shock ordering), calling `VarianceDecomp(shock=2, show=True)` computes statistics for structural shock 2 but plots the FEVD of variable 2, which are two different things that happen to share an index.

4 Tables

Table 1: *Input for SVAR class*

Name	Necessary?	Type	Domain	Default	Description
data	True	DataFrame or Array	Numerical data	-	DataFrame or Array with time series columns
variables	True	List	Columns of data or String names for array	-	Names of endogenous variables
lags	True	Integer	{1, 2, 3, ...}	-	Number of endogenous variable lags (k)
transformation	False	List	{None, 'diff', 'logdiff', 'log'}	None	Per-variable transformation: 'diff' for a first difference, 'logdiff' for a logarithmic first difference, 'log' for a level log transform, or None for no transformation
const	False	Integer	{0, 1, 2, 3}	1	Value 0 adds no deterministic term, value 1 adds a constant term, value 2 adds a constant and linear time trend terms and value 3 adds a constant, linear and squared time trend terms
rescaling	False	List	$\mathbb{R}$	1 per variable	Each variable's data is multiplied by its entry, applied uniformly across every transformation [†]
restrictions	False	String	{None, 'zeros', 'custom zeros'}	None	Type of linear restriction imposed on the VAR coefficients via the GLS estimator (2a)
R	False	Array	Binary mask (if restrictions ='zeros') or restriction matrix	None	Restriction / mask matrix used to build R in equation (2a) when restrictions is not None
method	False	String	{'short', 'long', 'signs', 'IV'}	'short'	Structural matrix B identification scheme (Section 2.4)
matrix_short	False	Array	{0, 1}	None	Zero-restriction mask for method ='short': 1 = free entry, 0 = restricted to zero. If given, B is solved by constrained optimization instead of Cholesky; if None, defaults to Cholesky
matrix_long	False	Array	{0, 1}	None	Zero-restriction mask for method ='long', applied to the long-run impact matrix $C = M_1 B$. If None, defaults to a lower-triangular C via Cholesky
matrix_signs	False	Array	{-1, 0, 1}	None	Sign restriction matrix; required when method ='signs' (Section 2.4.3)
iv	False	List or String	Column(s) of data or string name(s)	None	Instrument column name(s); required when method ='IV'
shocks	False	List	Shock names	variables	Names of the structural shocks, used in plot titles and legends
data_exogenous	False	DataFrame or Array	Numerical data	None	Exogenous regressors [‡]
variables_exogenous	False	List	Columns of data_exogenous or string names	None	Exogenous variable column names [‡]
DUM	False	String	{None, 'Q', 'M'}	None	Adds quarterly (3 dummies) or monthly (11 dummies) seasonal dummy regressors
impact	–	Float	$\mathbb{R}$	1.0	Stored on self and forwarded to a local variable inside <i>ImpulseResponse()</i> , but never passed on to <i>bootstrap.svar()</i> – the bootstrap IRF pipeline always effectively uses a unit shock [§]
steps	False	Integer	{1, 2, 3, ...}	60	Impulse response / variance decomposition horizon

[†] Applied uniformly across all four **transformation** branches as of v1.0.1 (previously not applied for 'diff'). [‡] Renamed from **EXOGdata/EXOG** in earlier drafts. [§] Flagged for wiring into the bootstrap pipeline in a future release; see Section 5.

Table 1 (continuation): *Input for SVAR class*

steps_signs	False	Integer	{1, 2, 3, ...}	1	Number of horizons over which sign restrictions on the IRFs are enforced (method='signs')
resampling	False	Integer	{1, 2}	1	Value 1 is for normal (i.i.d.) bootstrap resampling and value 2 is for wild bootstrap resampling
alpha	False	Float	(0,100)	32	Complement of the confidence level for the central bootstrap confidence band (e.g. 32 gives a 68% band, the 'many'-bands option additionally reports the 90%, 95%, 98% and 99% bands)
reps	False	Integer	{1, 2, 3, ...}	1000	Default number of bootstrap repetitions, used by <code>ImpulseResponse()</code> / <code>Forecast()</code> / <code>VarianceDecomp()</code> whenever their own reps argument is not given [†]
titles	False	List	Names for plotting	variables	Subplot titles for IRF, forecast and FEVD plots
horizon	False	Integer	{1, 2, 3, ...}	6	Forecast horizon
past	False	Integer	{1, 2, 3, ...}	12	Past periods shown alongside the forecast

[†] Renamed from **reps_default** in earlier drafts. Note that `bootstrap_svar()` itself defaults to **100** reps if called directly without this value – see the warning in Section 3.8.

Table 2: *Attributes cached internally when method='signs'*

Name	Type	Description
SVAR.IRL	Dict[int, List[Array]]	Raw accepted IRF draws per shock, shape (n_accept, steps, n_vars); consumed by <code>ImpulseResponse()</code>
SVAR.FEVDL	Dict[int, List[Array]]	Raw accepted FEVD draws per shock, shape (n_accept, steps, n_vars); consumed by <code>VarianceDecomp()</code>
SVAR.ir_median, ir_mean, ir_low/high/00/99/02/97/05/95/10/90/16/84/25/75	Dict[int, Array]	Shock-indexed IRF statistics computed once in <code>identify_signs()</code> ; each value has shape (steps, n_vars). Re-sliced by variable inside <code>ImpulseResponse()</code> on every call
SVAR.vd_median_signs, vd_mean_signs, vd_low/high/00/99/02/97/05/95_signs	Dict[int, Array]	Shock-indexed FEVD statistics, analogous to the <code>ir.*</code> attributes above. Kept under a "_signs" suffix so <code>VarianceDecomp()</code> can re-slice them by shock without being overwritten by its own output dicts

These are implementation details, not part of the primary public API – they exist so `ImpulseResponse()` and `VarianceDecomp()` can slice per-shock statistics computed once during `SVAR.S()` without re-running the accept/reject sampler on every call. As of v1.0.1, every key here is guaranteed to exist and be correctly shaped even if the sampler aborts after 100,000 rejections (zero-filled in that case), or if a particular shock ends up with zero accepted draws.

Table 3: *Output for SVAR.run() method*

Name	Type	Description
SVAR. Y	Array	Array of the endogenous variables time series that enter OLS
SVAR. X	Array	Array of the regressors that enter the OLS
SVAR. beta	Array	Array with the estimated coefficients
SVAR. beta_std	Array	Array with coefficients' standard errors
SVAR. Y_hat	Array	Array with the endogenous time series minus the residuals
SVAR. resid	Array	Array of residuals
SVAR. sigma_u	Array	Array of the variance-covariance matrix
SVAR. tstat	Array	Array of coefficient t-statistics
SVAR. pvalue	Array	Array of coefficient p-values ($df = T - k$ uniformly; see Section 3.3)
SVAR. ll	Float	Log likelihood
SVAR. aic	Float	Aikake information criterion
SVAR. bic	Float	Bayesian information criterion
SVAR. hqic	Float	Hannan-Quinn information criterion
SVAR. det	Float	Variance-covariance matrix determinant
SVAR. fpe	Float	Final prediction error, $\det(\hat{\Sigma}_u) \left(\frac{T+k}{T-k} \right)^n$ (implemented as of v1.0.1)
SVAR. n_params	Integer	Number of free parameters, R.shape [1] (new in v1.0.1)
SVAR. F	Array	Companion form coefficient matrix, built by SVAR.VAR() itself (not by estimate_var() ; see Section 3.3)
SVAR. B	Array	Structural error matrix (populated once SVAR.S() runs, e.g. inside SVAR.run() ; for method='signs' this is the <i>median</i> of the accepted set, not a single draw – consistent with the "Matrix B (median)" label printed by summary())
SVAR. n	Integer	Rejected-matrix count: draws violating the identification scheme (method='signs') or a fixed 0 otherwise
SVAR. m	Integer	Accepted-matrix count (method='signs' only; undefined for other methods)
SVAR. corr	List of Floats	Correlation between each instrument and its target variable (method='IV' only; 0 for other methods)
SVAR. structural	Boolean	True once SVAR.S() has run and B is populated
SVAR. summary	String	The full text report printed by SVAR.summary()

Table 4: *Input for SVAR.ImpulseResponse() method*

Name	Necessary?	Type	Domain	Default	Description
shock	True	Integer	$\{1, 2, \dots, n^*\}$	-	Number of the structural shock for which to calculate the IRF
show	False	Boolean	$\{\text{True}, \text{False}\}$	True	<i>True</i> shows the plot and <i>False</i> does not
bands	False	String	$\{\text{'pair'}, \text{'many'}\}$	'pair'	Value 'pair' shows the single central confidence band at the model's alpha level, value 'many' shows nested bands at 68, 90, 95 and 99%
reps	False	Integer	$\{1, 2, 3, \dots\}$	reps of the model	Bootstrap repetitions for this call; purely local, does not mutate self.reps [†]
steps	False	Integer	$\{1, 2, 3, \dots\}$	steps of the model	Horizon for this call; purely local, does not mutate self.steps [†]
impact	False	Float	$\mathbb{R}$	impact of the model	Accepted but not forwarded to the bootstrap pipeline; has no effect (see Table 1)

* Number of endogenous variables. [†] As of v1.0.1, overrides passed here are local to this call; earlier drafts permanently overwrote the corresponding **self** attribute, affecting later calls too.

Table 5: *Input for SVAR.VarianceDecomp() method*

Name	Necessary?	Type	Domain	Default	Description
shock	True	Integer	$\{1, 2, \dots, n^*\}$	-	Number of the structural shock to decompose [†]
show	False	Boolean	$\{\text{True}, \text{False}\}$	True	<i>True</i> shows the plot and <i>False</i> does not
bands	False	String	$\{\text{'pair'}, \text{'many'}\}$	'pair'	Stored on self.bands but does not affect the FEVD stacked-area plot, which always shows the deterministic self.vd_det (see Section 2.9)
steps	False	Integer	$\{1, 2, 3, \dots\}$	steps of the model	Horizon for this call; purely local
reps	False	Integer	$\{1, 2, 3, \dots\}$	reps of the model	Bootstrap repetitions for this call; purely local

* Number of endogenous variables. [†] **shock** is now a required positional argument, matching **ImpulseResponse()** – earlier drafts gave it a default of 1 here. The same index is also passed to **plot_variance_decomp_svar(self, shock)**, where it is reinterpreted as a *variable* index rather than a shock index (see Section 3.9).

Table 6: *Output for SVAR.ImpulseResponse() and SVAR.VarianceDecomp() methods*

Key	Type	Description
'point'	Dict of Arrays	Point estimate per variable: the bootstrap mean for method $\in \{\text{'short'}, \text{'long'}, \text{'IV'}\}$, or the median across the accepted sign-restriction draws for method ='signs'
'low' / 'high'	Dict of Arrays	Lower / upper bound of the central alpha -level band
0.5 / 99.5	Dict of Arrays	0.5th / 99.5th percentile across draws [†]
2.5 / 97.5	Dict of Arrays	2.5th / 97.5th percentile across draws
5 / 95	Dict of Arrays	5th / 95th percentile across draws

Each dictionary is keyed by variable name and holds an array of length **steps** with the corresponding statistic across horizons. [†] **ImpulseResponse()** **bug (method $\neq$ 'signs' only):** key **99.5** is currently mapped to the 97.5th-percentile array (duplicating key **97.5**) instead of the true 99.5th percentile, which is computed but discarded. Not present under **method**='signs', nor in **VarianceDecomp()**'s output (both correctly use the 99.5th percentile). See Section 2.

Table 7: *Input and output for SVAR.Forecast() method*

Name	Necessary?	Type	Domain	Default	Description
show	False	Boolean	{True, False}	True	<i>True</i> shows the plot and <i>False</i> does not
bands	False	String	{'pair', 'many'}	'many'	Same convention as in Table 4
horizon	False	Integer	{1, 2, 3, ...}	horizon of the model	Forecast horizon for this call; ignored under method ='signs' [†]
past	False	Integer	{1, 2, 3, ...}	past of the model	Historical periods shown alongside the forecast; ignored under method ='signs' [†]
reps	False	Integer	{1, 2, 3, ...}	reps of the model	Bootstrap repetitions for this call; unused under method ='signs' (no bootstrap is performed)

The output dictionary follows the same key structure as Table 6 ('point', 'low', 'high', 0.5, 2.5, 5, 95, 97.5, 99.5), with each array of length **past** + **horizon** instead of **steps**. Under **method**='signs' all nine dictionary entries are the *same* deterministic forecast path (no bootstrap, no dispersion), so 'low'/'high' and every percentile are identical to 'point'. [†] Under **method**='signs', `compute_forecast()` reads **self.horizon**/**self.past** directly rather than these local overrides – see Section 2.

Table 8: *Input and output for SVAR.stability() method*

Name	Necessary?	Type	Domain	Default	Description
show	False	Boolean	{True, False}	False	<i>True</i> additionally prints the raw eigenvalues and their moduli

Returns a single **Boolean**: *True* if every eigenvalue of the companion matrix **F** lies strictly inside the unit circle, *False* otherwise. Always prints a one-line PASSED/FAILED diagnostic regardless of **show**.

5 Known issues

This section collects every $\triangle$ -marked item from Sections 2–4 in one place, for triage before the next release. Items already resolved in v1.0.1 ($\checkmark$ boxes above) are not repeated here.

Location	Description
macroshock.py:: ImpulseResponse()	Non- 'signs' output dict maps key 99.5 to <code>self.ir_97</code> instead of <code>self.ir_99</code> (duplicating key 97.5); <code>self.ir_99</code> is computed but discarded.
macroshock.py:: ImpulseResponse()	<code>impact</code> override is computed locally but never passed to <code>bootstrap_svar()</code> ; the bootstrap IRF pipeline always uses <code>impact=1.0</code> regardless of <code>self.impact</code> or the per-call override.
macroshock.py::Forecast()	<code>horizon/past</code> overrides are silently ignored under <code>method='signs'</code> , since <code>compute_forecast()</code> always reads <code>self.horizon/self.past</code> . <code>self.fc_point</code> (non- 'signs' branch) is computed but never used by the output dict or the plot.
macroshock.py:: VarianceDecomp()	Each call re-runs a full bootstrap per shock; reporting all n shocks costs $n \times$ the necessary work. The new <code>reps</code> override lowers the cost per call but does not remove the redundancy.
macroshock.py::run()	No separate <code>VDbands</code> parameter: <code>showVD=True</code> reuses <code>IRbands</code> , which is largely inert for the FEVD plot anyway (see <code>plotting.py</code> below).
macroshock.py::S()	An unrecognized <code>method</code> does not raise; it leaves <code>structural=True</code> with no <code>self.B</code> , surfacing as an unrelated <code>AttributeError</code> later.
identification.py:: identify_signs	The <code>_high/_low</code> percentile convention is swapped (compensated on read in <code>macroshock.py</code>).
var.py::estimate_var	p -values use $df = T - k$ uniformly, not adjusted for the number of restrictions actually imposed via R .
data.py::YX	<code>model.R</code> has two different semantics depending on <code>restrictions='zeros'</code> vs 'custom zeros' .
bootstrap.py	Duplicates <code>data.py</code> 's X/Y construction logic inline instead of reusing it – the root cause that let the companion-matrix divergence above happen unnoticed.
plotting.py:: plot_variance_decomp_svar	The <code>shock</code> parameter indexes a <i>variable</i> , not a structural shock, despite sharing its name and value with <code>VarianceDecomp(shock=...)</code> 's genuinely shock-indexed statistics.